

# AI Engineer 2026 Takeaways

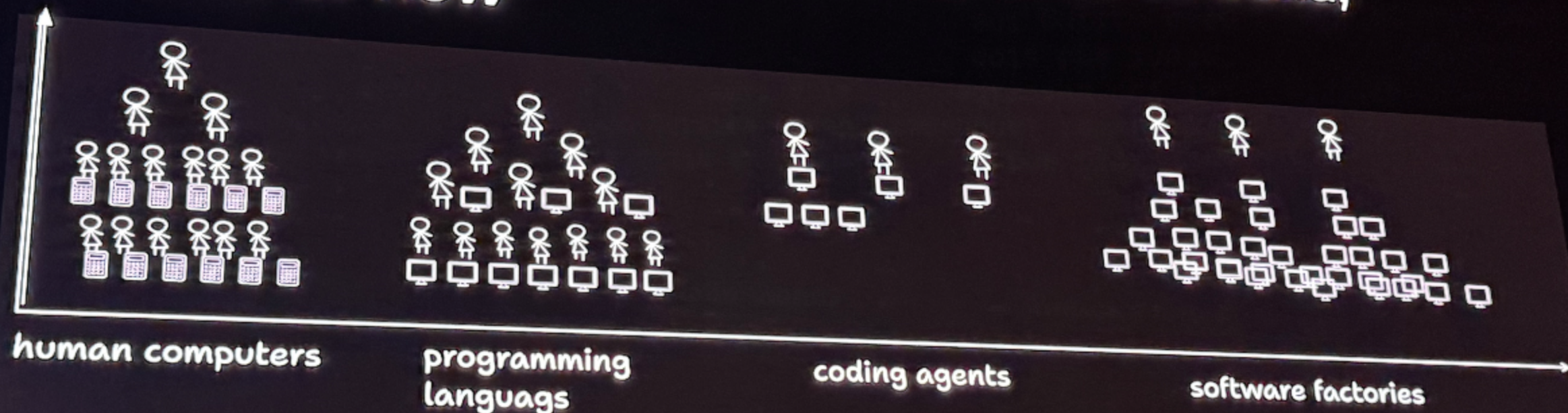
## From AI prompting to software factories

A practical playbook for using AI agents, keeping code safer, and getting more leverage.

---

The change is not better prompting. It is work that keeps moving, proves itself, and remembers what we meant.

# We move up, towards deciding *what* to build, instead of *how*



**EARLY 1940s**

Calculating ballistics trajectories by hand, wiring diagrams and patch cables.

**1950s - 2020s**

Compilers and high-level languages let you describe intent and stop hand-wiring the machine

**2023 - 2025**

Human is still at the desk, working one task at a time.

**NOW**

Engineers decide what to build, agents decide how

# The hard part moved from writing code to making judgment calls.

Code is getting cheaper. The hard part is choosing what to build, checking that it is safe, and improving the loop.

OLD BOTTLENECK

**Writing code**

NEW BOTTLENECK

**Choosing,  
checking,  
improving**

- What to build
- How much proof is enough
- What the factory must remember

# AI agents turn what you want into a process

Give the agent a clear goal, the tools it can use, and the proof you need before it starts writing code.



AI / Codex /  
Claude

**AI agents**

## **/goal: stop condition**

The goal says when the work is done.

## **Real app access**

Let the agent inspect the real UI when scripted checks are not enough.

## **Ask for proof before done**

Tests, screenshots, and notes show it worked.

# Tool choice can vary. Verification is the standard.

The operating model transfers across AI tools. The team habit is the same: define the work, give the tool room to act, and require proof before trust.

| Workflow                   | AI agent pattern              | Claude pattern                   |
|----------------------------|-------------------------------|----------------------------------|
| ✓ <b>One clear outcome</b> | /goal                         | /goal                            |
| ✓ <b>Recurring checks</b>  | Automation / heartbeat thread | /loop or scheduled task          |
| ✓ <b>Real app evidence</b> | Real-browser evidence         | No equivalent in tested terminal |
| ✓ <b>Parallel work</b>     | Worktrees or cloud tasks      | Worktrees or parallel sessions   |
| ✓ <b>Reusable habits</b>   | Skills, plugins, AGENTS.md    | Skills, hooks, CLAUDE.md         |

## Quick note

Claude terminal computer use was not available. Chrome-plugin workarounds exist, but they feel hacky.

If we can verify it, we can automate it. If we cannot verify it, we should keep humans closer.

# Run a lazy learner through TestLab with a real-browser agent

Use your voice. Let the agent click through TestLab, inspect the synthetic learner run, and come back with evidence.

## Voice prompt

/goal  
Run a 5-minute TestLab real-browser demo.  
Open TestLab -> Learner Automation Tests.  
Find a lazy or low-motivation synthetic learner run.  
Inspect learner, memories, notes, and chat.  
Capture tab-only Chrome screenshots; do not capture the desktop.  
Create a polished local HTML report with tab screenshots, evidence, blockers, and one follow-up eval.  
Do not create, delete, modify, or trigger runs without approval.  
End with the report path and next decision.

- Visible mouse-driven workflow.
- HTML report with tab-only screenshots.
- Safe boundary: inspect before acting.

# Real-browser automation is a bridge, not the destination

It helps us automate weak spots while our build, test, and release process catches up. It should not become a substitute for real CI/CD.

## Right now

An agent can click through brittle build or TestLab flows to collect evidence when a scripted path does not exist yet.

## Proper endpoint

GitHub Actions should run the build in a container, execute Playwright smoke tests, and publish artifacts.

## How we evolve

Use real-browser automation to learn the workflow. Then turn the repeatable parts into automated gates.

The hack should shrink as our trust and test coverage grow.



# Use factories where the ground is stable

Match autonomy to the code area, not to our enthusiasm.  
The operating mode should change as the proof gets stronger.

**Greenfield / isolated tools** Faster loops and more automation are reasonable.

**Well-tested MAS areas** Agents draft, CI gates, humans review by exception.

**Legacy / high-risk MAS areas** Small changes, stronger proof, human review stays close.



# Let agent threads coordinate the day

Keep a pinned Chief of Staff thread watching the signals. Then ask ticket, PR, deck, and review threads to pull the latest context instead of starting cold.

## Pinned context

Chief of Staff keeps today's Slack, PRs, pilot feedback, and blockers in one place.

## Cross-thread ask

Ask: "check today's Chief of Staff thread, then update this ticket with the next action."

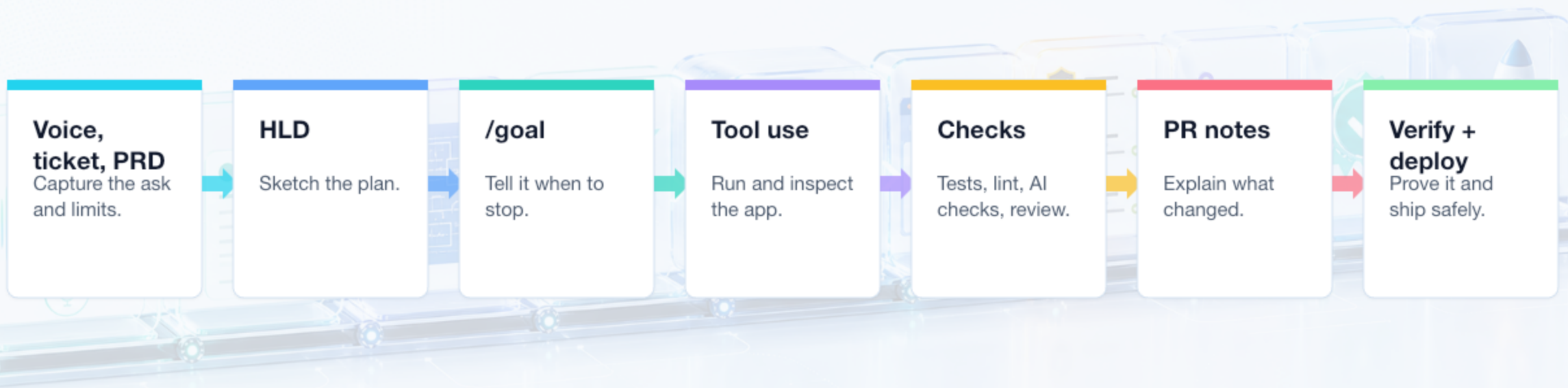
## Daily loop

A thread automation wakes up, checks signals, drafts next steps, and escalates decisions.

New habit: route context, then let the right thread act.

# Turn what you want into checked work

A good handoff is not a vague prompt. It tells the agent the goal, the tools, the checks, and what finished means.



# Define the modes

Do not do everything by hand or let agents run wild. Start with a factory you can watch.

## Lights-on factory

Humans stay close to the work.

- Agent assists, but humans make the important calls.
- Review happens before merge, deploy, or high-risk action.
- Best for new, risky, or poorly understood work.



### Monitored

Agents in the loop.  
Checks show progress.  
Humans decide.

## Lights-off factory

Signals go in; production-ready work comes out with minimal human touch.

- Agents plan, code, test, repair, and deploy through checks.
- Humans handle results, incidents, policy, and cost.
- Only safe after you can see what happened, roll back, and trust the tests.

# Schedules keep work alive without babysitting

We used to be the babysitter. Now the agent can keep checking the boring parts and only wake us when a person needs to decide.

## PR babysitter

Every hour: watch checks, comments, and merge conflicts. Fix safe conflicts or summarize blockers.

## Chief of Staff

Every 1-2 hours: surface unanswered Slack threads and draft replies for review.

## Pilot feedback loop

Monitor Slack. Turn real bugs into PRs, run checks, and post proof that it worked.

**Pilot feedback is the first mini software factory: Slack signal -> bug -> PR -> proof it worked.**

# Convert recurring work into loops we can observe

The loop is the repeated work. The goal says when to stop. The schedule decides when the loop wakes up.

## Bug loop

Absorb assigned bug -> draft PR -> test locally -> post verification.

## Inbox loop

Read morning inbox -> draft replies -> build the day's task list.

## Feedback loop

Pilot signal -> Slack -> product triage -> next ticket.

## HLD/PRD loop

Read draft -> find gaps -> propose changes -> prepare review.

# Build the workflow, not another prompt

When you keep giving the same prompt, turn it into a skill, plugin, or automation the team can reuse.

## 1. Capture

Interview the user before turning a successful workflow into a skill.

## 2. Package

Write down the inputs, checks, defaults, and examples so the behavior is reusable.

## 3. Share

After trust builds, publish the skill or plugin for the team.

# Start with humans watching, then let the loop do more

Do not jump straight to a lights-off factory. Start with small loops, watch what they produce, and expand only when the checks hold.

1

## Assigned bugs

The agent drafts PRs, tests locally, and posts proof that it worked.

2

## PR maintenance

Automations keep branches mergeable and surface blockers.

3

## Review checks

Repeated review comments become automated checks.

4

## Incident loops

Monitoring feeds focused fixes back into the factory.

**Trust comes from watching the loop work, not from declaring it trustworthy on day one.**

# Measure results, not activity

Busy agents can still waste time. Judge the factory by delivery speed, defects, whether it followed the process, and how much review it creates.

## Bad output metrics

- PRs merged per week
- Lines of code
- Time to first token
- Agent busy time

## Good result metrics

- Feature delivery time
- Defect rate stays stable
- Process followed by default
- Human review load shrinks

# Code review will soon be the bottleneck

As generated code volume rises, the old line-by-line review model turns engineers into full-time reviewers and burns attention.

## The old move

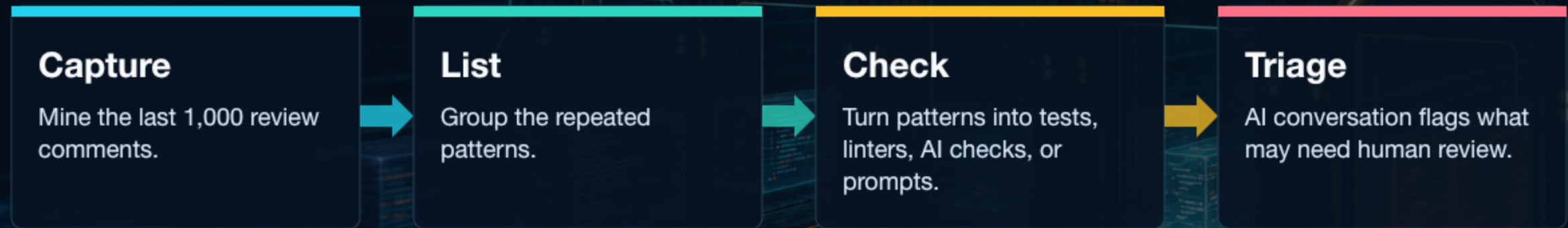
- Open diff, read line by line.
- Lose context across thousands of lines.

## The adaptation

- Talk through the diff with AI first.
- Ask what changes the goal, risk, or contracts.
- Send humans there, not to boilerplate.

# Code reviews: turn repeated review pain into automated checks

The useful thing is the review pattern list: repeated human comments turned into checks before the next reviewer spends attention.



**Review the goal, contracts, and risky paths, not the boilerplate.**

# Build the system that builds the system

Do not get stuck manually prompting agents one task at a time. Move daily work into small loops, check the proof, and let trust build.

## 1 Pick one daily pain point

Pilot bugs, PR babysitting, inbox triage, HLD review.

## 2 Build and verify the loop

Require tests, screenshots, checks, and a short proof note.

## 3 Let it do more gradually

When the loop behaves, schedule it and expand its scope.



## Daily challenge

Turn one daily task into a trusted factory loop.

The goal is not to babysit AI agents while trying to stay in control. It is to pick the right leverage points, set the checks, and watch the software-building process with confidence.

- Goals say what done means.
- Loops run recurring work.
- Checks keep leverage safe.

**Use AI as leverage to  
make big things happen**

